3. Implement `IClassFixture<WebApplicationFactory<T>>` in your xUnit test class, where `T` is a class in your real application's project. By convention, you typically use your application's `Program` class for `T`.

    - `WebApplicationFactory` uses the `T` reference to find the entry point for your application, running the application in memory, and dynamically replacing Kestrel with a `TestServer` for tests.
    - If you're using C# top-level statements and using the `Program` class for `T`, you need to make sure that the `Program` class is accessible from the test project. You can change the visibility of the automatically generated `Program` class by adding `public partial class Program {}` to your app.
    - The `IClassFixture<TFixture>` is an xUnit marker interface that tells xUnit to build an instance of `TFixture` before building the test class and to inject the instance into the test class's constructor. You can read more about fixtures at https://xunit.net/docs/shared-context.

4. Inject an instance of `WebApplicationFactory<T>` in your test class's constructor. You can use this fixture to create an `HttpClient` for sending in-memory requests to the `TestServer`. Those requests emulate your application's production behavior, as your application's real configuration, services, and middleware are all used.